Computer Graphics

第5回：アニメーションの基本 II

— 座標変換

情報学部 情報学科 情報メディア専攻
清水 哲也

今日の内容

1. 前回の復習と座標変換
2. 並進移動 `translate()`
3. 回転 `rotate()`
4. 拡大縮小 `scale()`
5. 座標変換の順序
6. 回転アニメーション
7. `pushMatrix()` / `popMatrix()`
8. 演習
9. まとめと次回

1. Introduction

前回の復習と座標変換

第4回の復習 | 位置を更新する

第4回では，図形の位置を変数で表しました．

```
x = x + v;
```

数式では，

$$x_{n+1} = x_n + v$$

です．

これまでの動かし方

図形の座標値そのものを変更していました.

```
ellipse(x, y, 80, 80);
x += vx;
y += vy;
```

ここでは，図形を描くたびに`x`と`y`を変えています.

今日の考え方

図形の座標値を直接変更する代わりに，

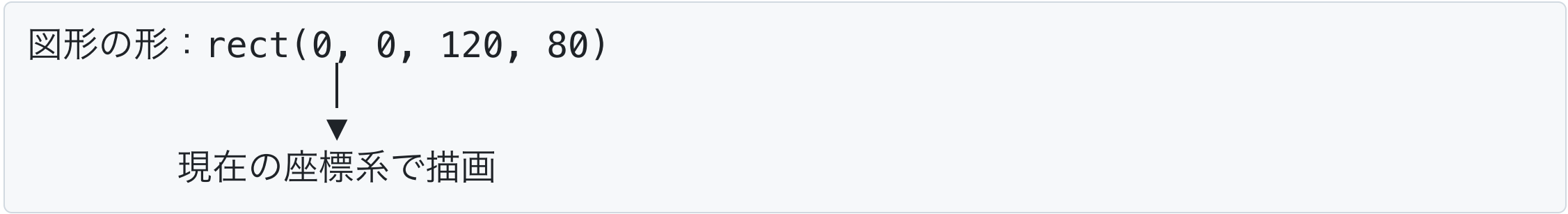
> 図形を描く座標系そのものを変換する

という方法を使います．

```
translate(tx, ty);
rotate(angle);
scale(sx, sy);
```

図形のコードは同じでも，座標系が変われば描画される位置や向きが変わります．

図形と座標系を分けて考える



図形を定義する座標と，画面上の位置・向きを分けて管理できます．

2. Translate

並進移動 `translate()`

`translate()` の基本

```
translate(tx, ty);
```

座標系の原点を，(t_x, t_y)だけ移動します．

その後に描く図形は，移動後の座標系を使います．

同じ `rect()` を別の位置に描く

```
rect(0, 0, 120, 80);

translate(250, 150);

rect(0, 0, 120, 80);
```

`rect()` の座標はどちらも同じですが，2つ目は移動後の原点から描かれます．

並進移動の数学

点の位置をベクトルで表すと，

$$\mathbf{p}' = \mathbf{p} + \mathbf{t}$$

です．

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} x \\ y \end{bmatrix} + \begin{bmatrix} t_x \\ t_y \end{bmatrix}$$

数学とProcessingの対応

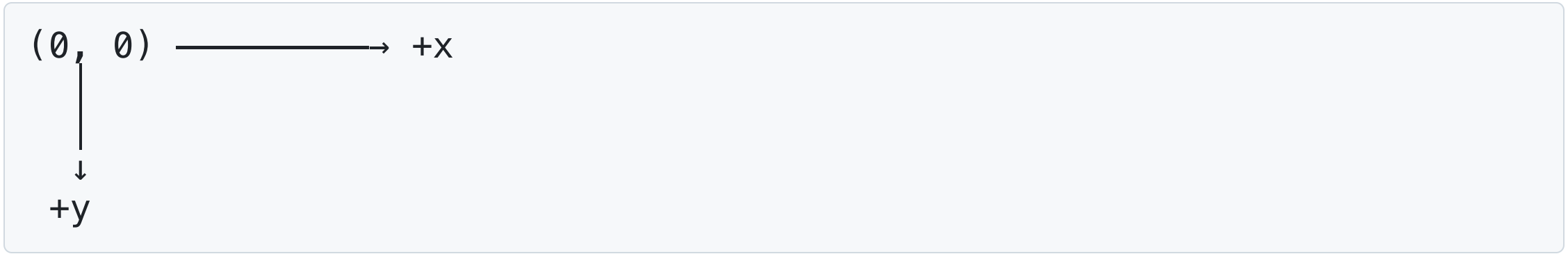
```
translate(tx, ty);
rect(x, y, w, h);
```

は，図形の点(x, y)に移動ベクトル(t_x, t_y)を加える操作と対応します．

```
変換前の点 + 移動ベクトル = 変換後の点
```

Processingの画面座標系

Processingでは，画面の左上付近が原点です．



xは右向き，yは下向きに増加します．

3. Rotate

回転 `rotate()`

`rotate()`の基本

```
rotate(theta);
```

座標系を角度θだけ回転します．

回転後に描いた図形は，回転した座標系の向きに従います．

回転の数学

点(x, y)は，次の規則で変換されます．

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

`rotate()`の内部では，この規則に対応する座標変換が行われています．

行列計算を目的にしない

今日の目的は，行列を手計算することではありません．

重要なのは，

- 図形の点が別の座標へ写される
- `rotate()` はその変換をまとめて実行する
- 変換後の座標系で図形を描く

という関係を理解することです．

角度の単位 | ラジアン

Processingの`rotate()`は，角度を**ラジアン**で受け取ります．

$$180^\circ = \pi\,\mathrm{rad}$$

```
rotate(PI / 4);         // 45度
rotate(radians(30));  // 30度
```

度からラジアンへの変換には`radians()`を使えます．

`PI` と `radians()`

```
float angle = radians(45);
rotate(angle);
```

`PI` は円周率πを表す定数です.

```
PI        // 180度
PI / 2    // 90度
PI / 4    // 45度
```

画面座標系での回転

Processingの画面ではy軸が下向きです．

そのため，正の角度を指定した`rotate()`は，画面上では**時計回り**に見えます．
数学でよく使うy軸が上向きの座標系では正の角度が反時計回りになるため，この違いに注意します．

角度は，Processingの画面座標系に対して指定されています．

4. Scale

拡大縮小 `scale()`

`scale()` の基本

```
scale(sx, sy);
```

座標系のx方向とy方向を，それぞれs_x，s_y倍します．

```
scale(2.0, 0.5);
```

x方向は2倍，y方向は0.5倍になります．

拡大縮小の数学

点(x, y)は，

$$x' = s_x x$$
$$y' = s_y y$$

に変換されます．

`scale()` と図形の大きさ

```
scale(1.5, 0.75);
rect(0, 0, 160, 100);
```

画面上では，おおよそ幅240，高さ75の図形として描かれます．

図形のコードを変えなくても，座標系の倍率を変えられます．

`scale()` の注意点

`scale()` は，後に描くすべての図形に影響します．

```
scale(2);
rect(0, 0, 100, 50);
ellipse(200, 100, 40, 40);
```

この2つの図形は，同じ拡大された座標系で描かれます．

必要な範囲だけに変換を適用する方法は，後で扱います．

5. Order

座標変換の順序

変換の順序は重要

次の2つは同じではありません．

```
translate(200, 150);
rotate(radians(30));
```

```
rotate(radians(30));
translate(200, 150);
```

実行する順序が変わると，図形の位置と向きも変わります．

なぜ結果が変わるのか

最初の変換は，次の変換が使う座標系も変えます．

1つ目の変換
↓
変化した座標系で2つ目の変換
↓
変化した座標系で図形を描く

したがって，後の`translate()`は，前の`rotate()`の影響を受けます．

TR と RT

変換を行列として表すと，

$$TR \neq RT$$

と書けます．ここでの記号の読み順は，行列をどちら側から点に作用させるかという約束に依存します．
この式を左から右へそのまま読むことはせず，

> 変換の順序を入れ替えると，一般に結果は同じにならない

という**非可換性**を表す式として理解します．これは，

> 平行移動してから回転する操作と，回転してから平行移動する操作は異なる

という意味です．

左右で比較する

```
pushMatrix();
translate(170, height / 2);  // 左右に置くための基準座標系
translate(80, 0);            // 比較する並進移動
rotate(radians(30));
rect(0, 0, 140, 60);
popMatrix();

pushMatrix();
translate(530, height / 2);  // 左右に置くための基準座標系
rotate(radians(30));
translate(80, 0);            // 比較する並進移動
rect(0, 0, 140, 60);
popMatrix();
```

同じ移動量・角度で順序を比較します．`pushMatrix()` / `popMatrix()`は左右の独立描画用です（詳細は7章）．

3DCGへの接続

2Dでの座標変換の順序は，3Dでも重要です．

3Dでは，

- 物体を移動する
- 物体を回転する
- 物体を拡大縮小する

という操作を組み合わせて，モデルの位置や向きを決めます．

6. Animation

回転アニメーション

位置更新から角度更新へ

第4回の位置更新は，

$$x_{n+1} = x_n + v$$

でした．

回転では，角度を状態として更新します．

$$\theta_{n+1} = \theta_n + \omega$$

ωは1フレームあたりの角速度です．

Processingとの対応

```
float angle = 0;
float omega = radians(2);

void setup() {
  size(600, 400);
  rectMode(CENTER);
}

void draw() {
  rotate(angle);
  angle += omega;
}
```

位置xを更新する代わりに，角度`angle`を更新しています．

正方形を回転させる

```
translate(width / 2, height / 2);
rotate(angle);
rectMode(CENTER);
rect(0, 0, 140, 140);
```

画面中央を座標系の原点に移動してから回転すると，正方形の中心を基準に回転できます．

回転アニメーションの全体

```
float angle = 0;
float omega = radians(2);

void setup() {
  size(600, 400);
  rectMode(CENTER);
}

void draw() {
  background(245);
  translate(width / 2, height / 2);
  rotate(angle);
  rect(0, 0, 140, 140);
  angle += omega;
}
```

7. Matrix Stack

`pushMatrix()` / `popMatrix()`

複数図形を別々に変換する

複数の図形を描くとき，変換をそのまま続けると，後の図形も影響を受けます．

```
translate(width / 2, height / 2);
rotate(angle);
rect(110, 0, 120, 50);

rotate(-angle * 2);
rect(-110, 0, 120, 50);
```

2つ目の図形は，1つ目の回転の影響を受けた座標系で描かれます．

`pushMatrix()` で保存する

```
pushMatrix();
translate(100, 0);
rotate(angle);
rect(0, 0, 120, 50);
popMatrix();
```

`pushMatrix()` は，その時点の座標変換状態を保存します．

`popMatrix()` で復元する

```
現在の座標系
     ↓
pushMatrix()
     ↓
translate / rotate / scale
     ↓
図形を描画
     ↓
popMatrix()
     ↓
保存時の座標系に戻る
```

`popMatrix()` を呼ぶと，保存した座標変換状態に戻ります．

2つの図形を独立して回転する

```
pushMatrix();
rotate(angle);
rect(110, 0, 120, 50);
popMatrix();

pushMatrix();
rotate(-angle * 2);
rect(-110, 0, 120, 50);
popMatrix();
```

それぞれの変換が，もう一方の図形に残りません．

座標変換の設計

複雑な図形を描くときは,

1. 基準となる座標系を決める
2. 必要なら`pushMatrix()`で保存する
3. `translate()` / `rotate()` / `scale()`を行う
4. 図形をローカル座標で描く
5. `popMatrix()`で復元する

という手順を意識します.

8. Exercise

演習

演習 I | 回転速度を変更する

回転アニメーションの`omega`を変更してください.

```
float omega = radians(2);
```

- `radians(1)`にするとどうなるか
- `radians(5)`にするとどうなるか
- 負の値にするとどうなるか

回転の向きと速さを観察してください.

演習2 | 回転中心を変更する

正方形の回転中心を，画面中央からずらしてください．

```
rectMode(CENTER);

translate(width / 2, height / 2);
rotate(angle);
rect(0, 0, 140, 140);
```

`translate()`の位置を変更すると，正方形が回転する中心がどう変わるでしょうか．
`rect()`は原点に置いたままにしてください．

演習3 | 複数図形を別々に回転する

`pushMatrix()` / `popMatrix()` を使って，2つの図形を描いてください．

- 2つの図形を異なる速度で回転させる
- 片方だけ回転方向を反対にする
- 図形の位置を左右に分ける

今日説明していない関数を必須にはしません．

9. Summary

まとめと次回

今日のまとめ

- 図形の座標値を変更する方法と，座標系を変換する方法がある
- `translate()` は座標系を移動する
- `rotate()` は座標系を回転する
- `scale()` は座標系の倍率を変える
- 変換の順序によって結果が変わる
- `pushMatrix()` / `popMatrix()` で変換を独立して管理できる

数学とProcessingの対応

数学的な状態	Processingの変数・関数
位置の更新 $x_{n+1} = x_n + v$	`x += v`
角度の更新 $\theta_{n+1} = \theta_n + \omega$	`angle += omega`
移動ベクトルの加算	`translate(tx, ty)`
回転行列による変換	`rotate(theta)`
座標の倍率変更	`scale(sx, sy)`

座標変換を使うときの手順

```
基準となる座標系を決める
        ↓
translate / rotate / scale
        ↓
図形をローカル座標で描く
        ↓
必要なら popMatrix() で戻す
```

図形を「どこにあるか」だけでなく，「どの座標系で描くか」で設計します．

次回 | 3Dモデルの描画とアニメーション

次回は，3Dモデルの描画とアニメーションを扱います．

2Dで学んだ，

- 移動
- 回転
- 拡大縮小
- 変換の順序

は，3Dでも重要な考え方になります．

`translate()` や `rotate()` を使って，3D空間の中のモデルを配置します．

参考：第5回サンプル

```
sample/sample_05_01_translate/
sample/sample_05_02_rotate/
sample/sample_05_03_scale/
sample/sample_05_04_transform_order/
sample/sample_05_05_rotation_animation/
sample/sample_05_06_push_pop/
```

各Sketchは，1つのディレクトリと1つのメイン`.pde`ファイルで構成しています．